ご提示いただいた track の難易度定義を拝見しました。

バンダイナムコスタジオの C++エンジニア職で**「上級」**が出題された場合、それは**「ゲーム開発の文脈における複雑性や計算機の効率的な使用」**を問うものになるでしょう。

純粋な競技プログラミングのような「高度な数理アルゴリズム」が出題される可能性もゼロではありませんが、それよりも**「ゲームエンジン/シミュレーション」**の専門知識と C++の深い理解を必要とする**「重実装」**や**「応用的なデータ構造」**が求められると想定されます。

これまでの議論を踏まえ、特に**「大規模なデータ」**と**「複雑性の高い操作を高速処理」**という上級の定義に合致する、ゲーム開発に特化した問題を 2 つ想定し、テストコード（解答例）と併せて提案します。

---

1. 衝突判定と空間分割の重実装（応用データ構造）

? 想定される分野と難易度

- **分野:** 空間分割（Quadtree/Octree）、衝突判定、メモリ効率
- **track 難易度:** 上級（複雑性の高い操作、重実装）
- **問われる能力:** 大規模なゲーム世界で数千のオブジェクトの衝突判定を**高速**に行うための、**データ構造の設計と実装能力。**

? 想定問題：Quadtree（四分木）を用いた高速な範囲探索

問題概要

2D の広大なフィールド（領域サイズ $S \times S$）にランダムに配置された**数千のユニット**（点または小さな矩形）が存在します。これらのユニット間の衝突判定の処理負荷を軽減するため、**Quadtree（四分木）**データ構造を実装してください。

以下の機能を満たす Quadtree クラスを実装し、指定された**検索範囲（AABB: Axis-Aligned Bounding Box）**内に存在するユニットを、**線形探索（O(N)）ではなく、高速に（O(log N)に近づけて）**抽出するメソッドを完成させてください。

1. 初期化時に最大深度と最大ノード容量を指定できること。
2. ユニットの**挿入**（ノードへの追加）を再帰的に行うこと。
3. 特定の矩形範囲内にあるユニットを**高速に検索**し、リストで返すこと。

C++ 解答例（Quadtree の基礎実装）

構造の定義

C++

```
#include <iostream>
#include <vector>
#include <memory> // unique_ptr for managed memory
#include <algorithm>

// 2D ポイント (ユニットの位置)
```

```
struct Point {
    float x;
    float y;
};

// 矩形 (AABB) - 領域や検索範囲
struct AABB {
    float x; // 左下 X 座標
    float y; // 左下 Y 座標
    float width;
    float height;

    // 矩形がポイントを含むかチェック
    bool contains(const Point& p) const {
        return p.x >= x && p.x < x + width &&
               p.y >= y && p.y < y + height;
    }

    // 矩形が他の矩形と交差するかチェック
    bool intersects(const AABB& other) const {
        return x < other.x + other.width &&
               x + width > other.x &&
               y < other.y + other.height &&
               y + height > other.y;
    }
};

class Quadtree {
private:
    AABB boundary;
    int capacity; // このノードが保持できる最大ユニット数
    std::vector<Point> points;

    // 子ノード (unique_ptr でメモリ管理の正確性を示す)
    std::unique_ptr<Quadtree> children[4];
    bool divided = false;

public:
    Quadtree(const AABB& boundary, int capacity)
        : boundary(boundary), capacity(capacity) {}

    // ノードを 4 つに分割
    void subdivide() {
        float subW = boundary.width / 2.0f;
        float subH = boundary.height / 2.0f;
        float x = boundary.x;
        float y = boundary.y;

        // 子ノードの境界を計算 (0: SW, 1: SE, 2: NW, 3: NE)
```

```
    children[0] = std::make_unique<Quadtree>(AABB{x, y, subW, subH}, capacity);
    children[1] = std::make_unique<Quadtree>(AABB{x + subW, y, subW, subH}, capacity);
    children[2] = std::make_unique<Quadtree>(AABB{x, y + subH, subW, subH}, capacity);
    children[3] = std::make_unique<Quadtree>(AABB{x + subW, y + subH, subW, subH}, capacity);

    divided = true;
}

// ポイントを挿入する
bool insert(const Point& p) {
    // 領域内に含まれない場合は挿入しない
    if (!boundary.contains(p)) {
        return false;
    }

    // 許容量未満で、まだ分割されていない場合、このノードに追加
    if (points.size() < capacity && !divided) {
        points.push_back(p);
        return true;
    }

    // 許容量を超えたら分割し、既存のポイントと新しいポイントを子ノードへ移動
    if (!divided) {
        subdivide();
        // 既存のポイントを子ノードへ移動
        for (const auto& existingP : points) {
            for (int i = 0; i < 4; ++i) {
                if (children[i]->insert(existingP)) break;
            }
        }
        points.clear(); // 親ノードのリストをクリア
    }

    // 新しいポイントを適切な子ノードへ再帰的に挿入
    for (int i = 0; i < 4; ++i) {
        if (children[i]->insert(p)) {
            return true;
        }
    }
    return false; // 理論的にはここに到達しない
}

// 範囲内のポイントを検索する (高速性の鍵)
void queryRange(const AABB& range, std::vector<Point>& found) {
    // 検索範囲がこのノードの領域と交差しない場合、即座に終了 (高速化)
    if (!boundary.intersects(range)) {
        return;
```

```
        }

        // このノードにポイントがある場合、検索範囲に含まれるかチェックして追加
        for (const auto& p : points) {
            if (range.contains(p)) {
                found.push_back(p);
            }
        }

        // 分割されている場合、子ノードに対して再帰的に検索
        if (divided) {
            for (int i = 0; i < 4; ++i) {
                children[i]->queryRange(range, found);
            }
        }
    }
};
//

void testQuadtree() {
    // 100x100 の領域、ノード容量は 4
    Quadtree qt(AABB{0, 0, 100, 100}, 4);

    std::vector<Point> insertedPoints;
    // 20 個のポイントを挿入
    for (int i = 0; i < 20; ++i) {
        Point p = {(float)(rand() % 100), (float)(rand() % 100)};
        qt.insert(p);
        insertedPoints.push_back(p);
    }
    std::cout << "20 個のポイントを Quadtree に挿入しました。" << std::endl;

    // 検索範囲を定義 (中央付近の小さな領域)
    AABB searchArea = {40, 40, 20, 20}; // x:40, y:40 から幅 20, 高さ 20
    std::vector<Point> results;

    // クエリ実行
    qt.queryRange(searchArea, results);

    std::cout << "検索範囲 AABB(40,40,20,20) 内に見つかったポイント数: " << results.size() << std::endl;
    // ... 結果の表示 ...
}

// int main() {
//     srand(time(0));
//     testQuadtree();
//     return 0;
// }
```

2. 複雑な状態遷移を持つAI システムの設計と制御（重実装・数理的発想）

? 想定される分野と難易度

- **分野:** 複雑な有限状態機械（FSM）とタイマー制御、遷移条件の優先度
- **track 難易度:** 上級（複雑性の高い操作、計算機の処理を効率的に使用）
- **問われる能力:** 複数のトリガーと制約（クールダウン、範囲、HP など）に基づき、**論理的に破綻しない**複雑な AI の行動ロジックを**オブジェクト指向的に**実装する能力。

? 想定問題：タワーディフェンスにおける複数攻撃モード AI

問題概要

タワーディフェンスゲームにおける防衛タワーユニットの AI ロジックを実装してください。このタワーは、以下の複数の状態とトリガーを持ちます。

1. **IDLE（待機）:** ターゲットを探す。
2. **TARGETING（照準）:** ターゲットに照準を合わせる（クールダウンあり）。
3. **ATTACKING（攻撃）:** 攻撃アクション中。
4. **RELOAD（リロード）:** リロードクールダウン中。

優先度に基づき、以下の条件を満たすように状態を遷移させる TowerAI クラスを完成させてください。

- **最優先:** リロード中は他の状態に遷移できない。
- **ターゲティング:** 攻撃範囲に入った**最も近い敵**をターゲットとし、ターゲティングクールダウンが完了していれば ATTACKING に遷移する。
- **攻撃:** 攻撃アクションが完了したら、すぐに RELOAD に遷移する。

C++ 解答例（複雑な FSM とタイマー制御）

構造の定義

C++

```
#include <iostream>
#include <map>
#include <functional> // std::function

// 敵の位置やターゲット距離などを管理する簡単な構造 (詳細は省略)
struct TargetInfo {
    float distance = 100.0f; // ターゲットまでの距離
    bool inRange = false;    // 攻撃範囲内か
};

enum class TowerState {
    IDLE,
    TARGETING,
```

```
    ATTACKING,
    RELOAD
};

class TowerAI {
private:
    TowerState currentState = TowerState::IDLE;
    TargetInfo target; // ターゲット情報

    // タイマー管理
    float reloadTimer = 0.0f;
    float targetingTimer = 0.0f;
    const float RELOAD_TIME = 2.0f;     // リロード時間
    const float TARGETING_TIME = 0.5f; // ターゲティング時間

    // 状態ごとの処理関数を格納するマップ (関数ポインタや std::function で実現)
    using StateHandler = std::function<void(float)>;
    std::map<TowerState, StateHandler> handlers;

public:
    TowerAI() {
        // 状態ハンドラを初期化
        handlers[TowerState::IDLE] = [this](float dt){ handleIdle(dt); };
        handlers[TowerState::TARGETING] = [this](float dt){ handleTargeting(dt); };
        handlers[TowerState::ATTACKING] = [this](float dt){ handleAttacking(dt); };
        handlers[TowerState::RELOAD] = [this](float dt){ handleReload(dt); };
    }

    void update(float deltaTime, const TargetInfo& currentTarget) {
        target = currentTarget; // ターゲット情報を更新

        // **最優先の制約チェック:** リロード状態の継続判定
        if (currentState == TowerState::RELOAD) {
            handleReload(deltaTime); // リロード中の処理は常に実行
            return; // 他の遷移は許可しない
        }

        // 現在の状態に応じた処理を実行
        if (handlers.count(currentState)) {
            handlers.at(currentState)(deltaTime);
        } else {
            // エラー処理、デフォルトで IDLE に戻す
            changeState(TowerState::IDLE);
        }
    }

    void changeState(TowerState newState) {
        if (currentState == newState) return;
```

```
        std::cout << "[FSM] Transition: " << stateToString(currentState)
                  << " -> " << stateToString(newState) << std::endl;
        currentState = newState;
    }

private:
    void handleIdle(float dt) {
        std::cout << "IDLE: Searching for target..." << std::endl;
        // ターゲットが範囲に入ったらターゲティングへ
        if (target.inRange) {
            changeState(TowerState::TARGETING);
        }
    }

    void handleTargeting(float dt) {
        targetingTimer += dt;
        std::cout << "TARGETING: Aligning aim... (" << targetingTimer << "s)" << std::endl;

        // ターゲティング時間が経過したら攻撃へ
        if (targetingTimer >= TARGETING_TIME) {
            changeState(TowerState::ATTACKING);
        }
        // ターゲットが範囲外に出たら IDLE に戻る
        else if (!target.inRange) {
            changeState(TowerState::IDLE);
        }
    }

    void handleAttacking(float dt) {
        // 攻撃処理（このフレームで攻撃が完了したと仮定）
        std::cout << "ATTACKING: Fire!" << std::endl;
        // 攻撃完了後、リロードへ
        changeState(TowerState::RELOAD);
    }

    void handleReload(float dt) {
        reloadTimer += dt;
        std::cout << "RELOAD: Cooldown... (" << reloadTimer << "s)" << std::endl;

        // リロード時間が経過したら IDLE へ
        if (reloadTimer >= RELOAD_TIME) {
            reloadTimer = 0.0f;
            changeState(TowerState::IDLE);
        }
    }
```

```
    // デバッグ用ヘルパー
    std::string stateToString(TowerState state) {
        switch (state) {
            case TowerState::IDLE: return "IDLE";
            case TowerState::TARGETING: return "TARGETING";
            case TowerState::ATTACKING: return "ATTACKING";
            case TowerState::RELOAD: return "RELOAD";
            default: return "UNKNOWN";
        }
    }
};

void testTowerAI() {
    TowerAI tower;
    float dt = 0.5f; // 0.5 秒刻みでシミュレーション

    // 1. 待機中にターゲットが入る
    tower.update(dt, TargetInfo{10.0f, false}); // IDLE
    tower.update(dt, TargetInfo{5.0f, true}); // IDLE -> TARGETING

    // 2. ターゲティング継続 (0.5s 経過)
    tower.update(dt, TargetInfo{5.0f, true});

    // 3. ターゲティング完了 (1.0s 経過) -> ATTACKING へ
    tower.update(dt, TargetInfo{5.0f, true}); // TARGETING -> ATTACKING -> RELOAD

    // 4. リロード開始 (2.0s 経過するまで他の処理は不可)
    tower.update(dt, TargetInfo{5.0f, true}); // RELOAD (0.5s)
    tower.update(dt, TargetInfo{5.0f, true}); // RELOAD (1.0s)
    tower.update(dt, TargetInfo{5.0f, true}); // RELOAD (1.5s)
    tower.update(dt, TargetInfo{5.0f, true}); // RELOAD (2.0s) -> IDLE へ

    // 5. ターゲットが範囲外に出る
    tower.update(dt, TargetInfo{50.0f, false}); // IDLE
}

// int main() {
//     testTowerAI();
//     return 0;
// }
```

この 2 つの問題は、単なるアルゴリズムの知識だけでなく、**「メモリ管理を意識した複雑なデータ構造の実装」と「複数の制約と優先度を持つゲームロジックのオブジェクト指向的な設計」**を問うものであり、「上級」の定義に最も合致すると言えます。